CHAPTER 2 · APPLICATION LAYER

2.1 Network Apps

Principles of Network Applications

Kurose & Ross, 8th Ed.

INTRO

What Does It Mean to Build a Network App?

Section 2.1: Introduction

Core task: write programs that run on different end systems and communicate over the network

What you DO need to write

Software that runs on end systems (hosts):

- Client program on the user's device (browser, app, email client…)
- Server program on the server host (web server, mail server, DNS server…)

Examples: Chrome ↔ Apache Web server
Netflix app ↔ Netflix CDN server

What you do NOT need to write

Software for network-core devices:

- Routers
- Link-layer switches

Core devices operate at the network layer and below — they never run application-layer code.

This design decision is what allows the rapid development of a vast array of applications.

Confining app software to end systems has enabled the Internet's explosive application growth

ARCHITECTURE

Network Application Architectures

Section 2.1.1: Application Architectures

The application architecture is the developer's choice — it is distinct from the (fixed) network architecture.

Two dominant paradigms:

Client-Server Architecture

- Always-on server with fixed, well-known IP address
- Clients may be intermittently connected, dynamic IP
- Clients never communicate directly with each other
- Server handles all requests
- Data centers used for massive scale (Google: 19 worldwide)

Peer-to-Peer (P2P) Architecture

- Minimal / no dedicated servers
- Direct communication between peer pairs
- Peers are user-owned desktops and laptops
- Each peer is both consumer AND provider
- Classic example: BitTorrent file sharing

ARCHITECTURE

Client-Server Architecture

Section 2.1.1: Application Architectures

The Server

Always on — never sleeps, never goes offline.

Fixed, well-known IP address — clients can always find it.

Services requests from potentially millions of clients simultaneously.

Never communicates directly with other clients.

The Client

May be intermittently connected.

May have dynamic IP addresses (changes on reconnect).

Does NOT communicate directly with other clients.

All communication is client → server → client.

The Data Center Problem

A single server host quickly gets overwhelmed by millions of clients. The solution: data centers — large collections of hosts acting as a powerful virtual server.

Google: 19 data centers worldwide · Hundreds of thousands of servers per center · Handles Search, YouTube, Gmail and more.

Cost: enormous investment in power, cooling, maintenance, and recurring bandwidth costs.

ARCHITECTURE

Peer-to-Peer (P2P) Architecture

Section 2.1.1: Application Architectures

Each peer is simultaneously a consumer AND a provider of service

Self-Scalability

Every new peer that joins adds capacity to the system — by uploading files to others while downloading.

More users = more capacity.

In client-server: more users = more load on server.

Cost Effectiveness

Minimal server infrastructure required.

No data center needed.

The peers do the work for free, on hardware they already own.

Ideal for resource distribution (files, video).

Challenges

Security: peers are user-controlled — hard to enforce correct behavior.

Performance: unpredictable availability.

Reliability: peers can leave at any time.

Example: BitTorrent — rarest-first + tit-for-tat.

Many modern apps use hybrid architectures — P2P for data transfer, client-server for directory/login

ARCHITECTURE

Client-Server vs. P2P: Key Trade-offs

Section 2.1.1: Application Architectures

Dimension	Client-Server	Peer-to-Peer
Server infrastructure	Required — data centers, always-on servers	Minimal / none
Scalability	Costly: each new user = more server load	Self-scaling: new users add capacity
Performance	Predictable if well-provisioned	Variable — depends on peer availability
Cost	High: power, cooling, bandwidth	Near zero for the provider
Control	Centralized: auth, billing, content policy	Decentralized: hard to enforce
Security / Reliability	Easy to manage centrally	Difficult — peers may misbehave
Example applications	Web, email, DNS, Netflix, Google Search	BitTorrent, early Skype voice calls

Processes Communicating

Section 2.1.2: Processes Communicating

It is not programs but PROCESSES that communicate across the network

What is a Process?

A process is a program running within an end system.

Processes on the SAME host communicate via interprocess communication (OS-managed).

We care about processes on DIFFERENT hosts — they communicate by exchanging messages across the network.

Client vs. Server Process

The CLIENT process is the one that INITIATES the session.
The SERVER process WAITS to be contacted.

Web: browser = client · web server = server
P2P: downloader = client · uploader = server

Note: in P2P, the same peer can be client in one session and server in another.

Concrete examples:

Application	Client Process	Server Process
Web	Browser process	Web server process
Email	Mail client (Outlook)	Mail server process
DNS	DNS client (resolver)	DNS server process
P2P	Downloading peer	Uploading peer

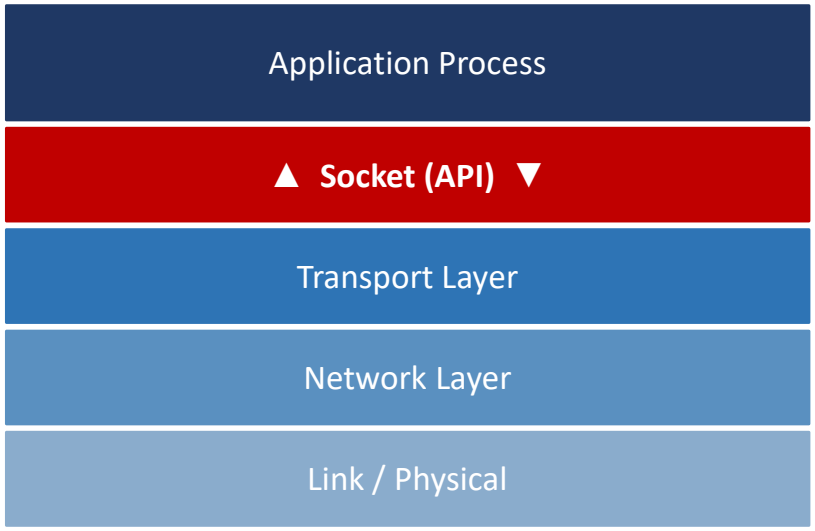
PROCESSES

The Socket — Interface to the Network

Section 2.1.2: Processes Communicating

A socket is the software interface (API) between the application process and the transport-layer protocol

The socket as API boundary:



What the developer controls:

✓ **Application side of socket**
Message format, logic, timing, encoding — completely up to you.

✓ **Choice of transport protocol**
TCP or UDP — you pick when you create the socket.

✓ **Some transport parameters**
Buffer sizes, maximum segment size, keepalive settings.

✗ **Network core routing**
How packets are forwarded inside the network — not your

Analogy: process = house · socket = door · network = transportation infrastructure outside the door

PROCESSES

Addressing Processes: IP Address + Port Number

Section 2.1.2: Processes Communicating

To deliver a message to a specific process on a specific host, you need two pieces of information:

1 IP Address

A 32-bit number that uniquely identifies the HOST.
Example: 142.250.185.36 (a Google server)

We study IP addresses in detail in Chapter 4.

2 Port Number

Identifies the specific PROCESS (socket) within the host.
A host runs many apps simultaneously — the port number tells the OS which one to deliver to.

Well-known port numbers (assigned by IANA):

80	443	25	53	21	22
HTTP (Web)	HTTPS	SMTP (Email)	DNS	FTP	SSH

Process address = IP address + Port number (e.g., 142.250.185.36 : 80 = Google's web server)

TRANSPORT

What Services Can a Transport Protocol Offer?

Section 2.1.3: Transport Services

Four dimensions along which transport services can be classified

1 Reliable Data Transfer

Guarantee that ALL data sent by the source process arrives correctly and completely at the destination — no missing bytes, no duplicates.

Critical for: email, file transfer, web, financial apps.
Acceptable to skip for: multimedia (voice, video) — these are loss-tolerant.

2 Throughput

Guarantee a minimum available bandwidth of r bits/sec.

Bandwidth-sensitive apps (Internet telephony at 32 kbps) need this — below the threshold the app fails.
Elastic apps (email, file transfer) adapt to whatever is available.

3 Timing

Guarantee that every bit arrives within a bounded delay (e.g., 100 ms).

Critical for interactive real-time apps: Internet telephony, multiplayer games, virtual environments, video conferencing.
Not needed for non-real-time apps.

4 Security

Encryption, data integrity, end-point authentication.

Prevents eavesdropping and tampering.
Critical for: e-commerce, banking, private communications, login credentials.

TRANSPORT

TCP Services

Section 2.1.4: Transport Services Provided by the Internet

TCP = Transmission Control Protocol · Connection-oriented · Reliable

Connection-Oriented Service

Before any application-level messages flow, TCP requires a HANDSHAKING procedure.

Client and server exchange control messages (SYN, SYN-ACK, ACK) to prepare for data transfer.

After handshaking: a TCP connection exists — a FULL-DUPLEX channel (both sides can send simultaneously).

When done: the connection must be explicitly torn down.

Reliable Data Transfer

Communicating processes can rely on TCP to deliver ALL data sent — without error, in proper order.

No missing bytes, no duplicate bytes.

If a packet is lost: TCP detects it (via ACK timeout) and retransmits automatically.

The application developer does not need to implement any error handling.

Congestion Control

TCP throttles the sender when the network becomes congested between sender and receiver.

This is for the GENERAL WELFARE of the Internet — not for the direct benefit of the application.

TCP attempts to give each connection a fair share of available network bandwidth.

Side effect: real-time apps dislike TCP because throttling introduces variable delay.

⚠ **TCP does NOT provide timing or throughput guarantees — only reliability and congestion control.**

TRANSPORT

UDP Services and Securing TCP with TLS

Section 2.1.4: Transport Services Provided by the Internet

UDP = User Datagram Protocol

Connectionless

No handshaking — processes start sending immediately.

Unreliable transfer

No guarantee a message reaches the destination.

Out-of-order delivery

Messages may arrive in a different order than sent.

No congestion control

Sender can pump data into the network at any rate.

No timing / throughput

Neither guaranteed.

Why use UDP? Speed + control + low overhead.
Best for: real-time multimedia, games, DNS.

TLS = Transport Layer Security

Neither TCP nor UDP provides encryption by default.
Data travels in cleartext — visible to any snooper on the path.

TLS adds to TCP:

Encryption

Data scrambled — unreadable in transit

Data Integrity

Any modification is detected

End-point Auth

Both parties verify each other's identity

Key: TLS is NOT a third transport protocol.
It is an application-layer enhancement of TCP (RFC 5246).
Developers include TLS code in their client and server programs.

TRANSPORT

TCP vs. UDP: What Each Provides

Section 2.1.4: Transport Services Provided by the Internet

Service	TCP	UDP
Connection setup	3-way handshake	None — connectionless
Reliable delivery	✓ Yes	✗ No
In-order delivery	✓ Yes	✗ No
Congestion control	✓ Yes	✗ No
Throughput guarantee	✗ No	✗ No
Timing guarantee	✗ No	✗ No
Security (built-in)	✗ No (use TLS)	✗ No
Overhead	High (headers, state)	Low (8-byte header)

TRANSPORT

Popular Applications: Protocol Choices

Section 2.1.4: Transport Services Provided by the Internet

Application	Protocol	RFC	Transport	Why That Choice?
Electronic mail	SMTP	RFC 5321	TCP	Email MUST arrive — data loss is unacceptable.
Remote terminal access	Telnet	RFC 854	TCP	Every keystroke and character must be delivered.
Web (HTTP)	HTTP/1.1	RFC 7230	TCP	Web pages must load completely and correctly.
File transfer	FTP	RFC 959	TCP	Every byte of the file must arrive intact.
Streaming multimedia	HTTP / DASH	—	TCP	Most CDN streaming uses TCP + adaptive bitrate (DASH).
Internet telephony	SIP / RTP	RFC 3261	UDP or TCP	Can tolerate loss but needs low delay. UDP preferred; TCP used as fallback if UDP is blocked.

TCP dominates because most apps need reliability · UDP chosen only when delay tolerance matters more than correctness

APP PROTOCOL

Application-Layer Protocols

Section 2.1.5: Application-Layer Protocols

An application-layer protocol defines how an application's processes pass messages to each other

An application-layer protocol must specify four things:

1 Types of messages

What kinds of messages exist?
Example: HTTP has request messages (GET, POST) and response messages (200 OK, 404 Not Found).

2 Syntax

What fields does each message contain and how are they delineated?
Example: HTTP request line format: METHOD URL VERSION\r\n

3 Semantics

What is the meaning of each field?
Example: HTTP status code 200 means success; 404 means the object was not found.

4 Rules

When and how does a process send a message and respond?
Example: a web server never sends a response unless it first receives a request from a client.

APP PROTOCOL

Open vs. Proprietary Protocols

Section 2.1.5: Application-Layer Protocols

An application-layer protocol is ONE component of a complete application — not the whole thing.

Open Protocols (defined in RFCs)

Specified in publicly available RFCs — known to all.

Any developer who follows the RFC can build a client or server that interoperates with any other conforming implementation.

Examples:
- HTTP (RFC 7230) — the Web
- SMTP (RFC 5321) — email
- DNS (RFC 1034) — domain names

Why it matters: Chrome can talk to Apache because both follow the HTTP RFC. Any email client works with any mail server.

Proprietary Protocols

Not publicly specified — controlled entirely by one company.

Client and server written by the same team; complete control over the design.

Independent developers cannot build interoperable implementations.

Examples:
- Early Skype voice protocol
- WhatsApp messaging protocol
- Netflix proprietary DASH variant

Trade-off: flexibility and control vs. interoperability and transparency.

Open protocols enable universal interoperability · Proprietary protocols give a single vendor total control

PREVIEW

Applications We Will Study in Chapter 2

Section 2.1.6: Network Applications Covered

2.2 The Web — HTTP

First application to capture the public's imagination (early 1990s). HTTP is straightforward and teaches core protocol concepts. Client-server, TCP, stateless.

2.3 Email — SMTP & IMAP

The Internet's first killer application. More complex than the Web — uses multiple cooperating protocols. Teaches how protocols interoperate.

2.4 DNS

The Internet's directory service — invisible to users but essential to everything. A beautiful distributed database implemented entirely at the application layer.

2.5 P2P — BitTorrent

Quantitative comparison of P2P vs. client-server scalability. BitTorrent's rarest-first and tit-for-tat mechanisms.

2.6 Video Streaming & CDNs

Video dominates Internet traffic. Adaptive streaming (DASH) + Content Distribution Networks delivering at massive scale.

Summary

2.1 Principles of Network Applications

- Building a network app = writing software for end systems only. No code runs in routers or switches.
- Two architectures: Client-Server (always-on, fixed IP, scalable via data centers) and P2P (self-scaling, peer-owned, cost-effective but harder to secure).
- Processes communicate via sockets — the API boundary between the app and the transport layer.
- Process address = IP address (identifies the host) + port number (identifies the process).
- Client = process that initiates a session · Server = process that waits to be contacted.
- 4 transport service dimensions: reliable data transfer, throughput, timing, security.
- TCP: connection-oriented, reliable, congestion-controlled. No timing/throughput guarantees.
- UDP: connectionless, unreliable, no congestion control — fast and lightweight.
- TLS: application-layer enhancement of TCP that adds encryption, integrity, and endpoint authentication.
- App-layer protocols specify: message types, syntax, semantics, and communication rules. Open (RFC) vs. proprietary.